

Question (May/April 2025)

Cloud kitchen aggregator app “Smoke & Spice” lists and promotes cloud kitchens, making them accessible to customers for online food ordering. It enables cloud kitchens to manage their menus, orders, and promotions while integrating with third-party delivery partners and payment gateways. The app challenges are handling high-order volumes, real-time order updates, seamless third-party integrations, and secure transactions.

Write a test plan for the following sections of the IEEE 829 test plan template for the above application:

- i) Scope of testing
- ii) Objectives
- iii) Risks
- iv) Strategy
- v) Approach

Answer

Test Plan for "Smoke & Spice" Cloud Kitchen Aggregator App (IEEE 829 Template)

i) Scope of Testing (What we will test and what we will NOT test)

What we WILL test:

- **Customer mobile app** – searching for cloud kitchens, viewing food menus, adding items to cart, applying promo codes, placing orders, tracking order status in real time.
- **Kitchen dashboard (cloud kitchen panel)** – updating food menu (add/edit/delete items), marking orders as "preparing", "ready for pickup", "dispatched", "delivered", creating discount promotions, viewing order history.
- **Order management system** – handling multiple orders at the same time, cancelling orders before preparation, auto-assigning orders to delivery partners.

- **Payment gateway integration** – testing UPI, credit/debit cards, net banking, wallets; testing success, failure, partial payment, refund scenarios.
- **Third-party delivery partner integration** – sending order details to delivery apps (like Zomato, Swiggy, Dunzo), receiving live tracking updates, handling delivery partner assignment and rejection.
- **Real-time updates** – order status changes should reflect instantly on customer app (e.g., "Order confirmed" → "Preparing" → "Out for delivery").
- **Performance under high load** – app should work smoothly when 1000+ customers order during lunch/dinner time.
- **Security testing** – customer login, payment details, order data should be encrypted; no unauthorized access.

What we will NOT test:

- Physical taste or quality of food (this is the kitchen's responsibility, not app testing).
- Delivery partner's internal systems (like their driver app's GPS bugs).
- Kitchen hardware (ovens, printers, etc.).
- Manual phone orders (only app-based orders are in scope).

ii) Objectives (What we want to achieve by testing)

1. **Correct order placement** – Every customer should place an order without errors. Cart total must match final payment.
2. **Real-time updates** – When the kitchen changes order status, the customer sees it within 2 seconds without refreshing.
3. **Payment security** – Payment details must be encrypted. Failed payments should show simple error messages (not technical errors).
4. **Smooth third-party integration** – Delivery partner gets correct customer details. If the delivery partner rejects, app finds another partner automatically.
5. **Handle high traffic** – App must work properly when 500+ users order at the same time (no crash or slow down).

6. **Proper error handling** – If payment fails, order is not created. If the delivery API is down, show a friendly message like "Please try again later".

iii) Risks (What can go wrong during testing or after launch)

Technical risks:

- **Third-party API downtime** – Payment gateway or delivery partner server may be unavailable during testing or peak hours, causing order failures.
- **Real-time update delay** – WebSocket or push notification service may lag, so kitchen updates order status but customer sees old status.
- **Database overload** – During high order volume, database queries may become slow, causing order duplication or missing orders.
- **Security vulnerability** – Hackers may try to steal payment data or change order amounts using API manipulation.

Project risks:

- **Time shortage** – Testing all integrations (multiple payment methods + multiple delivery partners) may take longer than planned.
- **Test environment mismatch** – Test servers may behave differently than live production servers, hiding some bugs.
- **Unclear requirements** – Business rules for promotions or cancellation may keep changing, leading to re-testing.

Operational risks:

- **Low kitchen cooperation** – Cloud kitchens may not provide test accounts or test menu data on time.
- **Delivery partner not available for testing** – Live testing with real delivery partners may require extra approvals.

v) Strategy (How we will test – which types of testing)

We will use different testing methods at different stages:

Testing Type	What we will do
Unit Testing	Developers test small pieces like "add to cart function" or "apply promo code function" before integration.
Integration Testing	Test connection between app → payment gateway, app → delivery partner, → kitchen dashboard. Check if data flows correctly.
System Testing	Test complete end-to-end flow: customer searches food → adds to cart → payment → kitchen receives order → kitchen updates status → delivery partner picks up → customer receives.
UI Testing	Check that buttons, screens, fonts, colors work properly on Android and iOS devices.
Real-time Testing	Manually change order status from kitchen panel and measure how fast customer app updates.
Performance/Load Testing	Use JMeter or LoadRunner to simulate 500, 1000, 2000 users ordering at same time. Check response time and error rate.
Security Testing	Check for SQL injection, broken authentication, exposed payment APIs. Follow OWASP guidelines.
Regression Testing	After fixing a bug, re-test old features to ensure nothing else broke.
User Acceptance Testing (UAT)	Let real cloud kitchen owners and real customers test the app before final launch.

Test Environment:

- Separate test server with dummy data.
- Test payment cards (Visa, Mastercard, UPI mock).
- Mock delivery partner simulator for initial testing.

Test Tools:

- Jira for bug tracking.
- Postman for API testing.
- JMeter for load testing.
- Appium for mobile automation (optional).

v) Approach (Step-by-step how we will test)

Step 1: Planning – Prepare test cases and test data (dummy kitchens, dishes, promo codes, customer accounts).

Step 2: Environment setup – Deploy app on test server, setup test payment gateway, connect to delivery partner sandbox.

Step 3: Functional testing – First test positive flows (order → pay → deliver). Then test negative flows (payment fail, delivery reject, wrong promo).

Step 4: Real-time testing – Kitchen changes order status. The customer sees an update within 2 seconds without a refresh.

Step 5: Integration testing – Test all payment methods (success/fail/refund). Test delivery API (send order → get tracking).

Step 6: Performance testing – Test with 500 users ordering together. Check response time (<3 sec) and error rate (<1%).

Step 7: Security testing – Try to change order amount or access another user's data – system must reject.

Step 8: Bug tracking – Log bugs in Jira. Developer fixes → tester retests → close bug.

Step 9: Sign-off – All critical bugs fixed, 95% tests passed, performance and security tests passed.

Step 10: Post-launch test – After live launch, retest 10 main features to confirm nothing broke.

Question (Oct/Nov 2025)

An online learning platform, *Knowledge*, provides interactive courses, video lectures and quizzes. It supports user roles such as students, instructors and administrators. The platform includes features like personalized learning recommendations, discussion forum, progress tracking and integration with third-party tools like payment gateways.

a) Write a test plan for the following sections of the IEEE 829 test plan template for the above application: [10]

- i) Scope of testing
- ii) Objectives
- iii) Risks
- iv) Strategy
- v) Approach

Answer

Test Plan for "Knowledge" Online Learning Platform (IEEE 829 Template)

i) Scope of Testing

What we WILL test:

- Student features – browse courses, watch video lectures, attempt quizzes, track progress.
- Instructor features – upload courses, create quizzes, reply to discussions.
- Admin features – manage users, approve courses, view reports.
- Personalized recommendations – system suggests relevant courses to students.
- Discussion forum – students can post questions, instructors can answer.
- Payment gateway integration – course purchase, subscription, refunds.
- Third-party tools – video streaming service, payment gateways.

What we will NOT test:

- Quality of video content (e.g., blurry video, poor audio).
- External payment gateway's internal security (only integration is tested).
- Student's internet connection or device hardware.

ii) Objectives

Objectives explain **why testing is done for the online learning platform.**

- **To ensure all features are working correctly:** All functions like login, course access, video lectures, quizzes, and payments should work without errors.
- **To provide a smooth user experience:** Students and instructors should be able to use the system easily without confusion, delays, or crashes.
- **To verify correctness of quizzes and progress tracking:** Quiz marks, results, and student progress reports should be accurate and updated properly.
- **To ensure secure payments and protect user data:** Payment transactions should be safe, and personal data (like login details) must be protected from unauthorized access.
- **To identify and fix bugs before release:** Testing helps find errors in the system so they can be corrected before users start using the platform.
- **To improve quality and reliability of the system:** The system should perform well under different conditions and give consistent results every time.

iii) Risks

1. **Video streaming failure:** Third-party video server may go down, stopping all lectures. Students will not be able to watch courses, leading to bad user experience and loss of trust.
2. **Payment gateway downtime:** If the payment gateway is down, students cannot purchase new courses during peak hours. This directly causes loss of revenue for the platform.
3. **Quiz data loss:** If a server crashes while a student is taking a quiz, all answers may be lost. Students will have to restart the quiz, causing frustration and wasted time.
4. **Wrong recommendations:** System may suggest irrelevant courses or courses already completed by the student. This makes the personalized learning feature useless and annoys users.
5. **Discussion forum spam**
Without proper spam filtering, users may post abusive content, ads, or irrelevant messages. This ruins the learning environment and may drive away genuine students.
6. **Progress mismatch:** Student completes a course but progress still shows as incomplete. This demotivates learners and creates confusion about what is actually finished.
7. **Role access violation:** A student may accidentally or intentionally access instructor features (like uploading courses). This can corrupt course data and create security issues.
8. **Poor performance during peak time:** When many students watch videos or take quizzes together, the platform may become slow or crash. This happens during exam time or new course launches.

iv) Strategy

- **Unit testing** – Test small functions like "calculate quiz score" and "update progress".
- **Integration testing** – Test connection between platform and payment gateway, video server.
- **System testing** – Test full flow: student login → buy course → watch video → take quiz → see progress.
- **UI testing** – Check screens on desktop, tablet, and mobile.
- **Performance testing** – Test with 1000 students watching videos simultaneously.
- **Security testing** – Check that one student cannot access another student's data.
- **Regression testing** – Re-test old features after every bug fix.

v) Approach (Step-by-step)

Step 1: Planning – Prepare test cases for login, course purchase, video play, quiz, forum, progress tracking.

Step 2: Environment setup – Setup test server, test payment gateway, mock video streaming service.

Step 3: Functional testing – Test positive flows (buy course → watch → take quiz → see progress). Test negative flows (payment fail → no access, wrong answer → low score).

Step 4: Role-based testing – Login as student (should only see courses). Login as instructor (should upload courses). Login as admin (should approve users).

Step 5: Recommendation testing – Complete 2 courses in "Python" → check if the system suggests more Python courses.

Step 6: Real-time progress testing – Watch half lecture → check progress shows 50%. Complete quiz → check marks added.

Step 7: Performance testing – Simulate 500 students watching videos at same time. Check buffering time and server load.

Step 8: Security testing – Try to access another student's course or progress – system must block.

Step 9: Bug tracking – Log bugs in Jira. Developer fixes → retest → close bug.

Step 10: Sign-off – All critical bugs fixed, 95% tests passed, performance and security okay.